Programming Language 구현과제1 보고서

20221867 이채은

[Python으로 구현한 Recursive Descent Parser]

```
import re

class Parser:
    def __init__(self):
        self.vars = {'x': 0, 'y': 0, 'z': 0}
        self.relops = ['==', '!=', '<', '>', '<=', '>=']
        self.answers = []

    def match(self, expected):
        if self.code[self.index] == expected:
            self.index += 1
        else:
            raise SyntaxError

    def last_match(self, expected):
        if self.code[self.index] != expected:
            raise SyntaxError

    def parse(self, code):
        self.code = code.split()
        self.index = 0
        self.code_len = len(self.code)-1
        try:
            self.program()
```

```
            except:
                print(">> syntax error!!")

    # <program> → {<statement>}
    def program(self):
        self.answers.clear()
        while self.index < len(self.code):
            self.statement()
        if self.answers:
            result = ['TRUE' if item is True else 'FALSE' if item is False else item for
item in self.answers]
            print('>>', *result)

    # <statement> → <var> = <expr> ; | print <var> ;
    def statement(self):
        token = self.code[self.index]
        if token == 'print':
            self.index += 1
            var = self.code[self.index]
            self.index += 1
            if var not in self.vars:
                raise Exception
            self.answers.append(self.vars[var])
            if self.index < self.code_len:
                self.match(";")
            else:
                self.last_match(";")

        elif token == ';' :
            self.index += 1
```

```
        else:
            var = token
            if var not in self.vars:
                raise Exception
            self.index += 1
            self.match('=')
            value = self.expr()
            self.vars[var] = value
            if self.index < self.code_len:
                self.match(";")
            else:
                self.last_match(";")

    # <expr> → <bexpr> | <aexpr>
    def expr(self):
        token = self.code[self.index]
        next_token = self.code[self.index+1]
        if token.isdigit() and next_token in self.relops:
            return self.bexpr()
        else:
            return self.aexpr()

    # <bexpr> → <number> <relop> <number>
    def bexpr(self):
        left = int(self.code[self.index])
        self.index += 1
        op = self.code[self.index]
        self.index += 1
        right = int(self.code[self.index])
```

```
            self.index += 1
            if op == '==':
                return left == right
            elif op == '!=':
                return left != right
            elif op == '<':
                return left < right
            elif op == '>':
                return left > right
            elif op == '<=':
                return left <= right
            elif op == '>=':
                return left >= right

    # <aexpr> → <term> {( + | - ) <term>}
    def aexpr(self):
        result = self.term()
        while self.index < len(self.code) and self.code[self.index] in ('+', '-'):
            op = self.code[self.index]
            self.index += 1
            term = self.term()
            if op == '+':
                result += term
            elif op == '-':
                result -= term
        return result

    # <term> → <factor> {( * | / ) <factor>}
    def term(self):
        result = self.factor()
```

```
            while self.index < len(self.code) and self.code[self.index] in ('*', '/'):
                op = self.code[self.index]
                self.index += 1
                factor = self.factor()
                if op == '*':
                    result *= factor
                elif op == '/':
                    result /= factor
            return int(result)

        # <factor> → <number>
        def factor(self):
            token = self.code[self.index]
            self.index += 1
            if token.isdigit():
                return int(token)
            else:
                return self.vars[token]

# Main 함수
if __name__ == "__main__":
    parser = Parser()
    while True:
        code = input('>> ')
        if code == "terminate":
            break
        parser.parse(code)
```

[Python코드에서 사용한 주요 함수와 리스트, etc.]

전체적으로는 Parser 클래스와 main 역할을 하는 함수를 만들었습니다. Enter를 누르면

자동으로 다음 입력으로 넘어가게 됩니다. 그리고 terminate를 입력하면 프로그램이 종료됩니다.

-self.vars: x, y, z의 값을 구분하고 저장합니다. 초기값은 모두 0으로 설정했습니다.

-self.relop: 비교 및 관계 연산자 모음을 만들어 나중에 비교하기 쉽게 만들었습니다.

-self.answers: 올바른 입력문에 대한 출력값 모음입니다.

-match(): 현재 인덱스의 토큰이 내가 기대하는 토큰과 맞다면 인덱스에 +1을 합니다.

-last_match(): 현재 인덱스가 마지막 인덱스일 경우 +1은 하지 않고 기대하는 토큰이 맞는지 검사합니다.

-parse(): 입력을 공백을 기준으로 구분합니다. 원래대로라면 숫자도 자리마다 따로 계산해야 하지만 코딩의 용이성을 위해 하나의 토큰으로 구분하여 code 리스트에 넣어줍니다

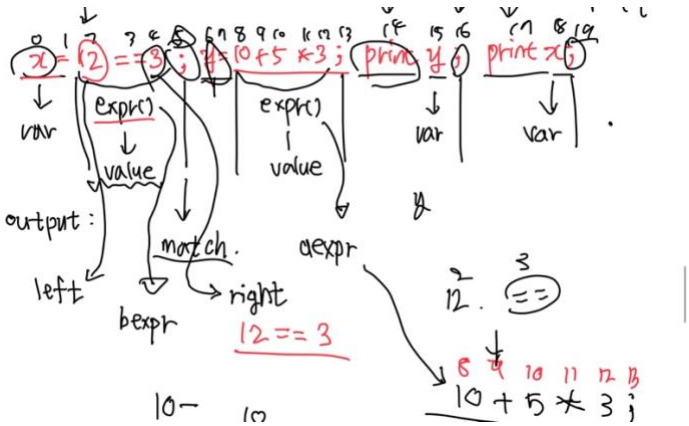
-program(): <statement>를 0번 이상 실행하고 결과를 출력합니다. Bool값이 존재한다면 TRUE와 FALSE 대문자 처리 후 출력되게 합니다.

-statement() : code 리스트에 저장된 토큰으로 분석을 시작합니다. print일 때, ;일 때, else의 경우로 처리를 했고, EBNF에 맞게 다음 토큰들을 처리하게 했습니다.

-expr(): 숫자로 시작하면서 next_token이 관계, 비교 연산자인 경우에는 <bexpr>을, 아닌 경우에는 <aexpr>로 넘어가게 했습니다.

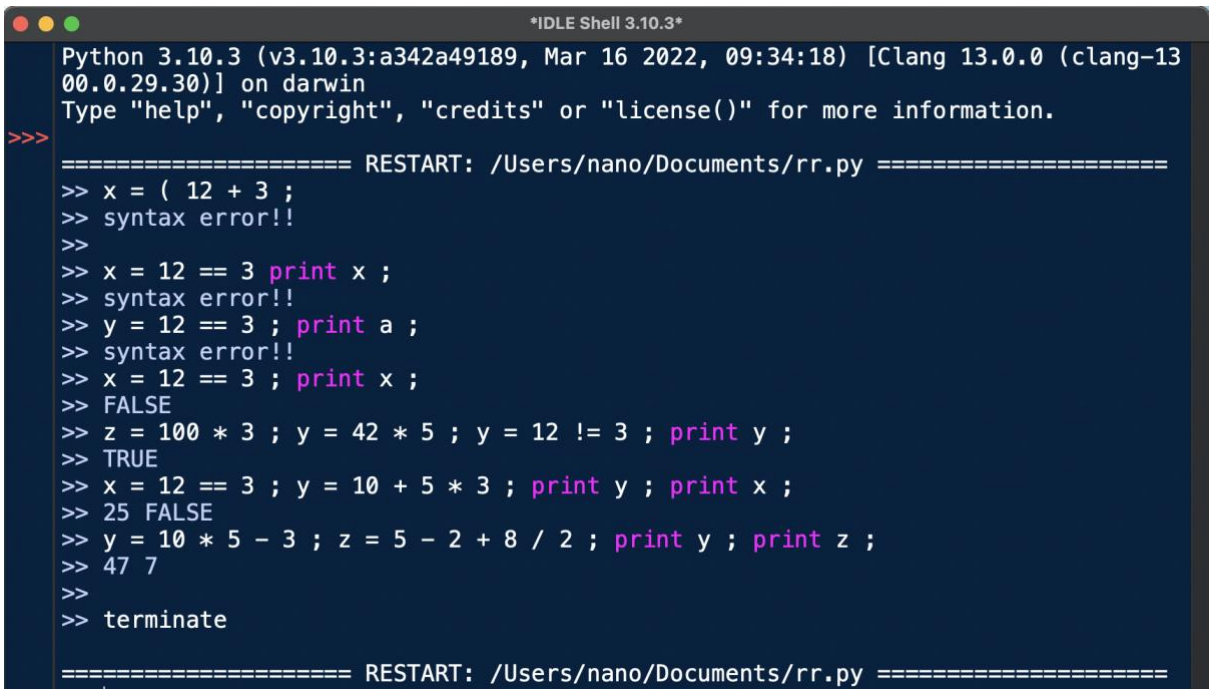
-bexpr(): <bexpr> → <number> <relop> <number>가 잘 동작되도록 했습니다. Left와 op, right로 토큰을 받아 처리하고 return하도록 했습니다.

-aexpr(), term(), factor(): 제일 까다로운 부분이었습니다. aexpr()은 <term>을 호출, term()은 <factor>를 호출, factor()는 토큰이 숫자이면 int로 변환하여 return하게 해서 이 과정이 반복되도록 했습니다. 즉, op와 result, index로 계산이 원활하게 이루어지도록 했습니다.

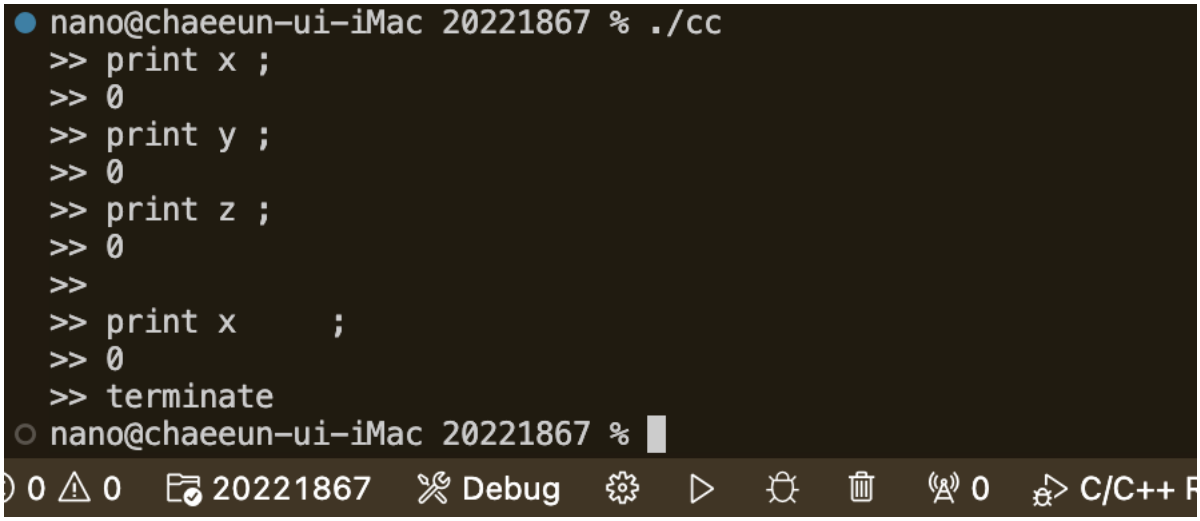


[실행 결과 스냅샷]

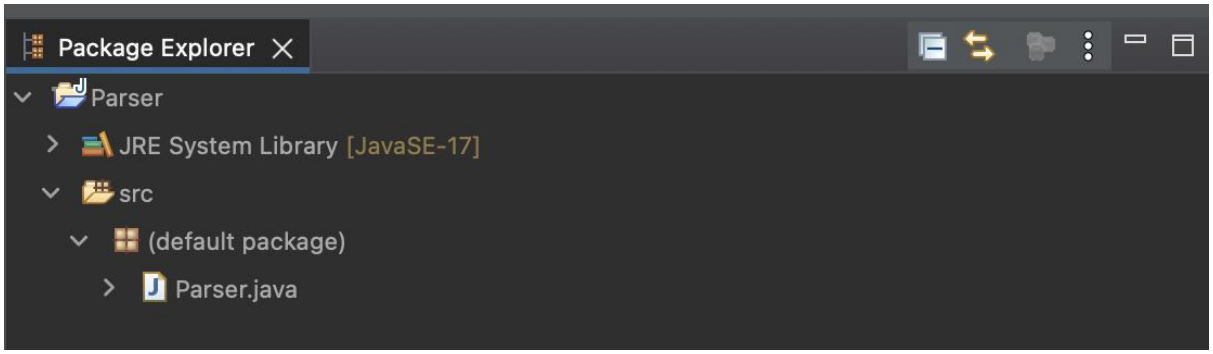
1. Python_IDLE



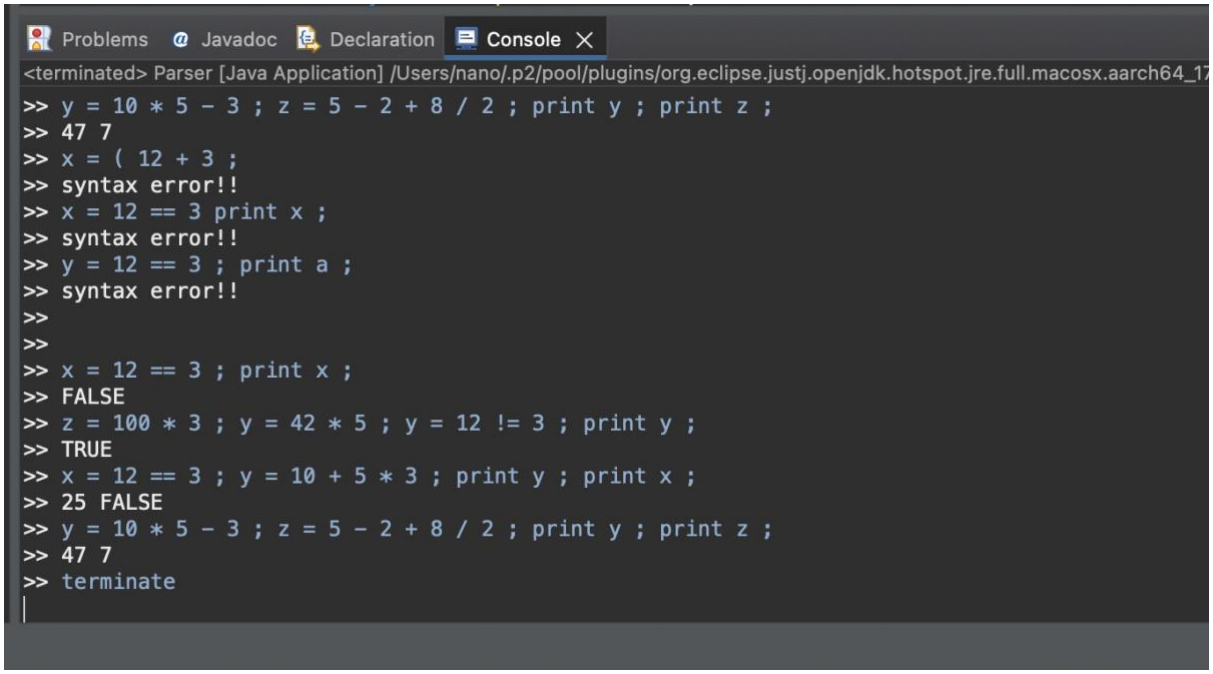
2. C_gcc → 구현 못함



3. Java_Eclipse


Package Explorer
Parser
JRE System Library [JavaSE-17]
src
(default package)
Parser.java

-이렇게 놓고 실행


Problems
Javadoc
Declaration
Console
<terminated> Parser [Java Application] /Users/nano/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.aarch64_17
>> y = 10 * 5 - 3 ; z = 5 - 2 + 8 / 2 ; print y ; print z ;
>> 47 7
>> x = (12 + 3 ;
>> syntax error!!
>> x = 12 == 3 print x ;
>> syntax error!!
>> y = 12 == 3 ; print a ;
>> syntax error!!
>>
>>
>> x = 12 == 3 ; print x ;
>> FALSE
>> z = 100 * 3 ; y = 42 * 5 ; y = 12 != 3 ; print y ;
>> TRUE
>> x = 12 == 3 ; y = 10 + 5 * 3 ; print y ; print x ;
>> 25 FALSE
>> y = 10 * 5 - 3 ; z = 5 - 2 + 8 / 2 ; print y ; print z ;
>> 47 7
>> terminate